

DATE : 29.09.2026

DT/NT : DT

LESSON : LINUX

**SUBJECT: FILTERS AND SHELL
OPERATORS**

SESSION : 7

BATCH : B 424-020DE

AWS-DEVOPS



TECHPRO
EDUCATION



techproeducation.com



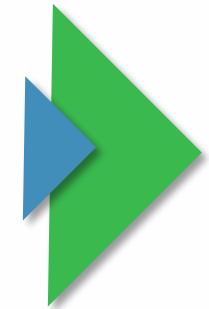
+1 (585) 304 29 59



Table of Contents

- ▶ stdin, stdout, stderr
- ▶ Redirection and Pipes
- ▶ Filters
- ▶ Commands: cat, tee, grep, cut, tr, wc, sort, uniq, comm
- ▶ Sed Commands
- ▶ Shell Operators, Exit Status & Special Characters





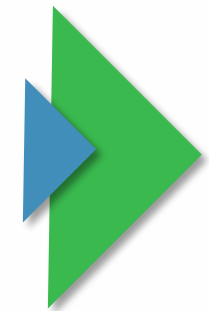
stdin, stdout, stderr



stdin, stdout, stderr



command > output.txt | command 2> error.txt | command > output.txt 2> error.txt



Filters



Filters



➤ Filter

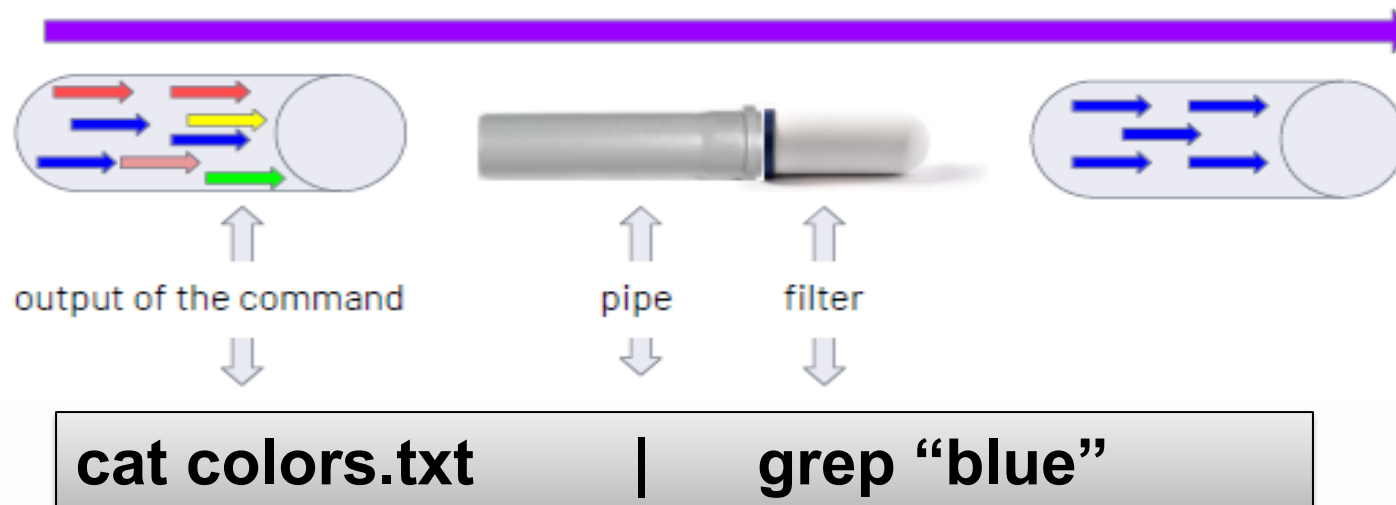
A filter reads input, performs an operation, and produces output. Filters are commonly combined with pipes.

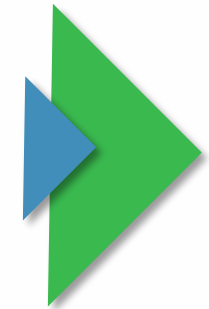
Pipe (|)

A pipe sends the stdout of one command to the stdin of the next command.

Example:

grep "blue" colors.txt





Commands



Filters Commands

cat

Displays file contents or copies stdin to stdout.

Avoid unnecessary cat in pipelines:

cat /var/log/app.log | grep ERROR

Prefer:

grep ERROR /var/log/app.log

cat is useful for displaying or combining files.



```
linuxuser@techproeducation: ~$ cat days.txt
monday
tuesday
wednesday
thursday
friday
saturday
sunday
linuxuser@techproeducation: ~$ cat days.txt | cat
monday
tuesday
wednesday
thursday
friday
saturday
sunday
linuxuser@techproeducation: ~$ cat days.txt | cat | cat
monday
tuesday
wednesday
thursday
friday
saturday
sunday
linuxuser@techproeducation: ~$
```

Filters Commands



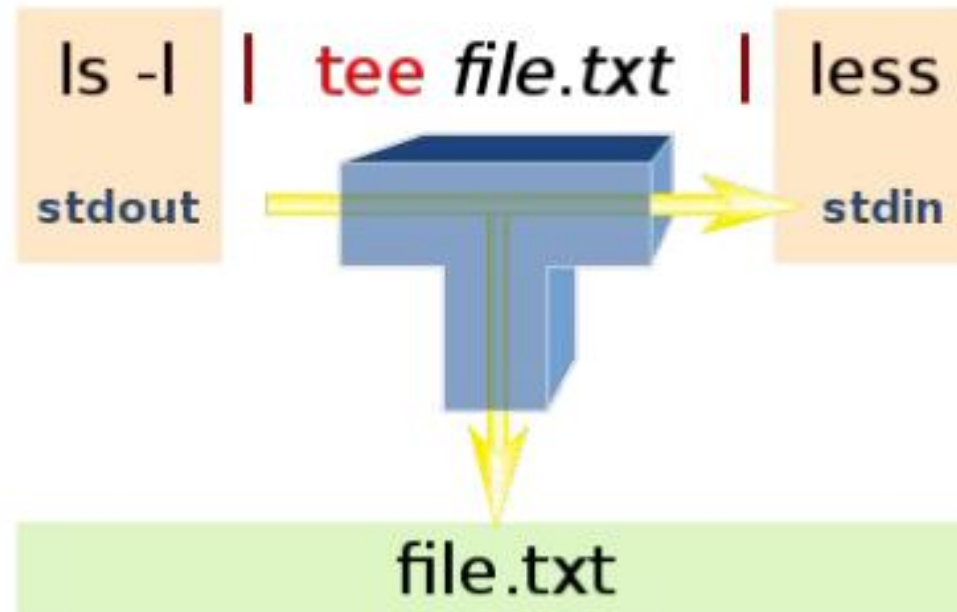
tee

Reads stdin and writes the same data to stdout and a file.

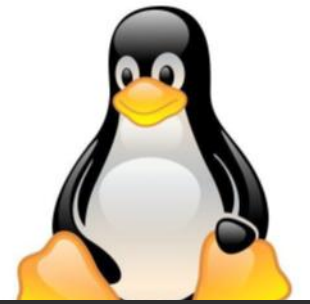
```
echo "Installing nginx" | tee install.log
```

Append instead of overwrite:

```
echo "Next step" | tee -a install.log
```



Filters Commands



grep

Searches text and prints lines that match a pattern.

```
grep "ERROR" app.log
```

```
grep -i "error" app.log
```

```
grep -v "INFO" app.log
```

```
grep -n "ERROR" app.log
```

```
grep -c "ERROR" app.log
```

```
grep "ERROR" app.log | wc -l
```

```
linuxuser@techproeducation: X + v - □ X
linuxuser@techproeducation:~$ cat tennis.txt
Roger Federer - İsviçre
Rafael Nadal - İspanya
Novak Djokovic - Sırbistan
Serena Williams - Amerika Birleşik Devletleri
Venus Williams - Amerika Birleşik Devletleri
Andre Agassi - Amerika Birleşik Devletleri
Steffi Graf - Almanya
Bjorn Borg - İsveç
Martina Navratilova - Amerika Birleşik Devletleri (Eski Çekoslovakya)
Pete Sampras - Amerika Birleşik Devletleri
linuxuser@techproeducation:~$ cat tennis.txt | grep Almanya
Steffi Graf - Almanya
linuxuser@techproeducation:~$ |
```

Filters Commands



cut

Selects fields or character ranges from each input line.

Syntax:

cut -d',' -f3 countries.csv

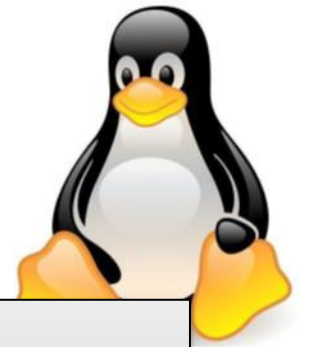
Example with /etc/passwd:

cut -d: -f1 /etc/passwd

cut -d: -f1,7 /etc/passwd

```
linuxuser@techproeducation: ~$ ls -l
total 21280
-rw-rw-r-- 1 linuxuser linuxuser      57 Dec  3 11:23 days.txt
-rw-rw-r-- 1 linuxuser linuxuser       6 Dec  3 11:31 file.txt
-rw-rw-r-- 1 linuxuser linuxuser 21765972 Nov 30 11:03 latest-hugo.zip
drwxrwxr-x 2 linuxuser linuxuser    4096 Dec  2 11:12 linux_lesson
drwxrwxr-x 2 linuxuser linuxuser    4096 Dec  1 10:40 permission
drwx----- 3 linuxuser linuxuser    4096 Dec  2 10:53 snap
-rw-rw-r-- 1 linuxuser linuxuser     374 Dec  3 11:34 tennis.txt
linuxuser@techproeducation:~$ ls -l | cut -d ' ' -f3
linuxuser
linuxuser
linuxuser
linuxuser
linuxuser
linuxuser
linuxuser
linuxuser@techproeducation:~$ |
```

Filters Commands



tr

Translates, replaces, or deletes characters.

```
echo "hello world" | tr 'a-z' 'A-Z'
```

```
echo "hello    world" | tr -s ' '
```

```
echo "abc123" | tr -d '0-9'
```

```
linuxuser@techproeducation: ~$ cat days.txt
monday
tuesday
wednesday
thursday
friday
saturday
sunday
linuxuser@techproeducation: ~$ cat days.txt | tr 'day' 'yad'
monyard
tuesyard
weynesyard
thursyard
friyard
saturyard
sunyard
linuxuser@techproeducation: ~$

linuxuser@techproeducation: ~$ cat days.txt
monday
tuesday
wednesday
thursday
friday
saturday
sunday
linuxuser@techproeducation: ~$ cat days.txt | tr '\n' ' '
monday tuesday wednesday thursday friday saturday sunday
linuxuser@techproeducation: ~$
```

Filters Commands



wc counts lines, words, bytes, and characters.

wc -l file.txt → **lines**

wc -w file.txt → **words**

wc -c file.txt → **bytes**

wc -m file.txt → **characters**

```
linuxuser@techproeducation: ~$ cat days.txt
monday
tuesday
wednesday
thursday
friday
saturday
sunday
linuxuser@techproeducation: ~$ wc days.txt
 7  7 57 days.txt
linuxuser@techproeducation: ~$ wc -l days.txt
7 days.txt
linuxuser@techproeducation: ~$ wc -w days.txt
7 days.txt
linuxuser@techproeducation: ~$ wc -c days.txt
57 days.txt
linuxuser@techproeducation: ~$ |
```

Filters Commands



sort

Sorts input lines. By default, sorting is lexicographic.

sort -r file.txt → reverse order

sort -f file.txt → case-insensitive

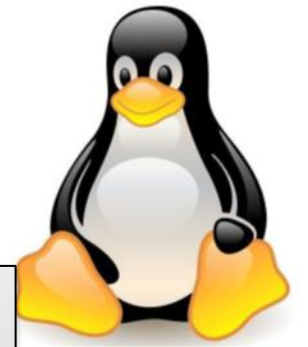
sort -n numbers.txt → numeric order

sort -k2 users.txt → sort by field 2

sort -t',' -k2 data.csv

```
linuxuser@techproeducation: ~$ cat days.txt
monday
tuesday
wednesday
thursday
friday
saturday
sunday
linuxuser@techproeducation: ~$ sort days.txt
friday
monday
saturday
sunday
thursday
tuesday
wednesday
linuxuser@techproeducation: ~$ |
```

Filters Commands



uniq

Removes adjacent duplicate lines. It does not sort the input.

sort names.txt | uniq

Or use:

sort -u names.txt

Duplicate lines must be adjacent for uniq to remove them.

```
linuxuser@techproeducation: ~$  
linuxuser@techproeducation: ~$ cat days.txt  
monday  
tuesday  
wednesday  
thursday  
friday  
saturday  
sunday  
monday  
sunday  
friday  
linuxuser@techproeducation: ~$ sort days.txt | uniq  
friday  
monday  
saturday  
sunday  
thursday  
tuesday  
wednesday  
linuxuser@techproeducation: ~$
```

Filters Commands



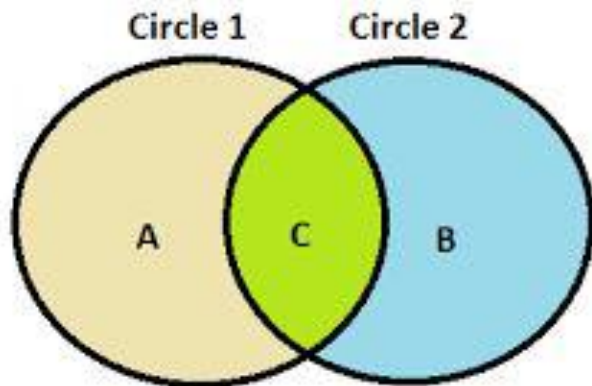
comm

Compares two sorted files and displays three columns:

1. Lines unique to file 1
2. Lines unique to file 2
3. Lines common to both

comm file1.txt file2.txt

comm -12 file1.txt file2.txt → common lines only



```
linuxuser@techproeducation: ~$ cat name2
dennis
jane
john
marc
mauro
mia
linuxuser@techproeducation: ~$ cat name3
aden
dennis
jane
mauro
mia
thuram
linuxuser@techproeducation: ~$ comm name2 name3
      aden
      dennis
      jane
john
marc
      mauro
      mia
      thuram
linuxuser@techproeducation: ~$
```

Exercise 1



1. Create countries.csv:

Country,Capital,Continent

USA,Washington,North America

France,Paris,Europe

Canada,Ottawa,North America

Germany,Berlin,Europe

2. Using filters:

A. Extract Continent. B. Remove the header.

C. Sort. D. Remove duplicates. E. Save as countries.txt.

3. Display countries.txt.

Suggested pipeline:

```
tail -n +2 countries.csv | cut -d',' -f3 | sort | uniq > countries.txt
```

Exercise 2 - AWS/DevOps Log Analysis



Create app.log with these lines:

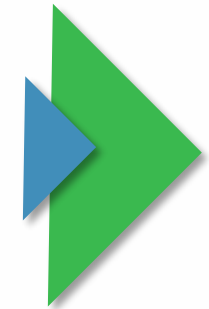
INFO Application started
INFO Database connected
ERROR Database connection failed
INFO Retrying connection
ERROR Timeout connecting to database
WARNING High memory usage
ERROR API request failed

Tasks:

1. Find ERROR lines.
2. Count errors with grep and wc.
3. Extract the error message.
4. Sort and show unique errors.
5. Save the final result to errors.txt.

Example:

```
grep "ERROR" app.log | cut -d' ' -f2- | sort | uniq > errors.txt
```



Sed Commands



Sed Commands



sed (Stream Editor)

Reads text streams and can transform or filter them.

Preview a replacement:

```
sed 's/old_text/new_text/' sed.txt
```

Edit the file in place:

```
sed -i 's/old_text/new_text/' sed.txt
```

Create a backup while editing:

```
sed -i.bak 's/old_text/new_text/' sed.txt
```

Sed Commands



Text Replacement

Preview:

```
sed 's/old_text/new_text/' sed.txt
```

Modify:

```
sed -i 's/old_text/new_text/' sed.txt
```

Backup:

```
sed -i.bak 's/old_text/new_text/' sed.txt
```

```
● [ec2-user@ip-172-31-19-189 ~]$ cat sed.txt
eski metin
● [ec2-user@ip-172-31-19-189 ~]$ sed -i 's/eski/yeni/' sed.txt
● [ec2-user@ip-172-31-19-189 ~]$ cat sed.txt
yeni metin
○ [ec2-user@ip-172-31-19-189 ~]$
```

Shell Operators, Exit Status & Special Characters



Shell Operators, Exit Status & Special Characters



Shell list operators:

- ; sequential commands
- & background execution
- && continue only after success
- || continue only after failure

Exit status:

\$? previous command's exit status

Special syntax:

comments | \ escape / line continuation

Semicolon (;)

; Semicolon

Runs commands sequentially, regardless of whether the previous command succeeds or fails.

mkdir test ; echo "Next command"

Compare with &&, where the second command depends on success.



```
linuxuser@techproeducation: ~$ cat name1
thuram
jane
mia
aden
dennis
mauro
linuxuser@techproeducation: ~$ cat name2
dennis
jane
john
marc
mauro
mia
linuxuser@techproeducation: ~$ cat name1; cat name2
thuram
jane
mia
aden
dennis
mauro
dennis
jane
john
marc
mauro
mia
linuxuser@techproeducation: ~$
```

Ampersand (&)



& Background execution

Runs the command in the background and returns the shell prompt.

sleep 20 &

Check jobs:

jobs

Wait for background jobs:

wait

```
linuxuser@techproeducation: ~$ sleep 20 &
[1] 653
linuxuser@techproeducation: ~$
[1]+  Done                  sleep 20
linuxuser@techproeducation: ~$
```

- Look at the above snapshot, command "sleep 20 &" has displayed a message after 15 seconds.
- Meanwhile, in the shell prompt, we can write any other command.

Dollar Question Mark (\$?)



\$? Exit Status

\$? is a special parameter containing the exit status of the most recently executed command.

0 normally means success.

A non-zero value means the command did not succeed.

ls /etc

echo \$?

ls /does-not-exist

echo \$?

```
linuxuser@techproeducation: X + - □ X
linuxuser@techproeducation:~/linux_lesson$ ls
Techproeducation.txt file1 file2 file3 myfile
linuxuser@techproeducation:~/linux_lesson$ echo $?
0
linuxuser@techproeducation:~/linux_lesson$ rmdir *
rmdir: failed to remove 'Techproeducation.txt': Not a directory
rmdir: failed to remove 'file1': Not a directory
rmdir: failed to remove 'file2': Not a directory
rmdir: failed to remove 'file3': Not a directory
rmdir: failed to remove 'myfile': Not a directory
linuxuser@techproeducation:~/linux_lesson$ echo $?
1
linuxuser@techproeducation:~/linux_lesson$ |
```

Double Ampersand (&&)



&& Logical AND

The second command runs only when the first command succeeds.

mkdir deployment && echo "Directory created"

Useful in automation and CI/CD pipelines.

```
linuxuser@techproeducation: X + v - □ X
linuxuser@techproeducation:~$ cd linux_lesson/ && ls
Techproeducation.txt file1 file2 file3 myfile
linuxuser@techproeducation:~/linux_lesson$ |
```

Double Vertical Bar (||)



|| Logical OR

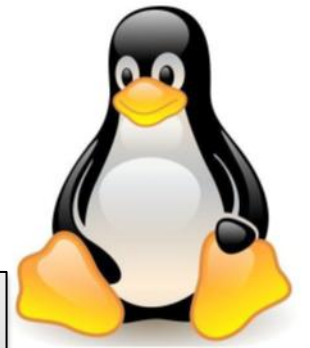
The second command runs only when the first command fails.

mkdir deployment || echo "Could not create directory"

Useful for fallback actions and error handling.

```
linuxuser@techproeducation: ~$ cat days.txt || echo "techproeducation" ; echo instructor
monday
tuesday
wednesday
thursday
friday
saturday
sunday
monday
sunday
friday
instructor
linuxuser@techproeducation: ~$ catt days.txt || echo "techproeducation" ; echo instructor
Command 'catt' not found, did you mean:
  command 'cat' from deb coreutils (8.32-4.1ubuntu1)
  command 'ratt' from deb ratt (0.0~git20190123.9e77a6d-1)
  command 'catm' from deb mescc-tools (1.3-2)
  command 'catty' from deb node-catty (0.1.0-1)
Try: apt install <deb name>
techproeducation
instructor
linuxuser@techproeducation: ~$ |
```

Combining && and ||



Combining && and ||

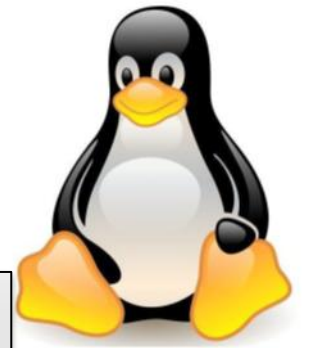
These operators can create simple success/failure branches.

systemctl is-active nginx && echo "Running" || echo "Not running"

Useful in shell automation and troubleshooting.

```
linuxuser@techproeducation: ~$ cat days.txt
monday
tuesday
wednesday
thursday
friday
saturday
sunday
monday
sunday
friday
linuxuser@techproeducation: ~$ rm days.txt && echo it worked! || echo it failed!
it worked!
linuxuser@techproeducation: ~$ rm days.txt && echo it worked! || echo it failed!
rm: cannot remove 'days.txt': No such file or directory
it failed!
linuxuser@techproeducation: ~$
```

Pound Sign (#)



Comments

Text after # is treated as a comment and is ignored by the shell.

Check whether nginx is running

systemctl is-active nginx

Comments document scripts without changing execution.

```
linuxuser@techproeducation: ~$ mkdir test # create directory
linuxuser@techproeducation: ~$ cd test    # change directory
```

Escaping Special Characters (\)



\ Escape Character

A backslash can prevent the shell from interpreting the next character specially.

echo "Price: \\$100"

Use it when a special character must be treated literally.

```
linuxuser@techproeducation: ~$ echo this is * symbol
this is file.txt latest-hugo.zip linux_lesson name1 name2 name3 permission
snap tennis.txt test symbol
linuxuser@techproeducation: ~$ echo this is \* symbol
this is * symbol
linuxuser@techproeducation: ~$
```

End of Line Backslash (\)



\ Line Continuation

A backslash at the end of a line continues the command on the next line.

```
echo "This is a long command" \  
    "continued on another line"
```

The shell treats the continued lines as one command.

```
linuxuser@techproeducation: X + v  
linuxuser@techproeducation:~$ echo This command line \  
> is split in three \  
> parts  
This command line is split in three parts  
linuxuser@techproeducation:~$ |
```

Exercise 3 - Control Operators



1. Search for `techproeducation.txt` in the current directory.
2. If it exists, display its contents.
3. If it does not exist, print: Too early!
4. Create `techproeducation.txt` containing: Congratulations.
5. Repeat the check.

Challenge: solve it with `&&` and `||`.

`find . -name "techproeducation.txt" && cat techproeducation.txt || echo "Too early!"`